BRIEF INTRODUCTION

1. Protein Modeling Protein modeling is the process of predicting the three-dimensional structure of a protein based on its amino acid sequence. Because a protein’s function is dictated by its shape, knowing the structure is the "Holy Grail" for understanding diseases.

Common Approaches:

- Homology (Comparative) Modeling: This is the "copy-paste" method. If we know the structure of a protein that is very similar to our unknown one, we use it as a template to build a model.
- Fold Recognition (Threading): If there isn't a direct match, we try to "thread" the sequence through a library of known protein folds to see which one fits best.
- Ab Initio (De Novo) Modeling: This is the hard way. It uses physics and chemistry laws to predict the shape from scratch without any templates. Programs like AlphaFold have revolutionized this area using AI.

2. Protein Docking If modeling builds the protein, docking tests how that protein interacts with others. It is a search for the best "fit" between two molecules—like finding the right key for a lock.

How it Works:

- Sampling: The computer explores thousands of orientations and positions of a "ligand" (a drug or another protein) relative to the target protein.
- Scoring: Each position is given a score based on energy. A "good" score means the molecules are attracted to each other and fit snugly without clashing.

Why this matters Imagine you want to design a drug to stop a virus.

- First, you use modeling to figure out what the virus's surface proteins look like.
- Then, you use docking to test thousands of digital drug molecules to see which one "docks" perfectly into the virus protein to switch it off.

PRACTICAL SBI15 AND SBI16 - RIGID BODY DOCKING

The goal of this practical is to use rigid-body docking to predict the structure of a proteinprotein complex from the coordinates of the unbound subunits, based on geometry complementarity and FFT-based search.

Why predicting protein-protein interactions? Most of protein functions in biological processes involve the interaction with other biomolecules and the formation of specific protein-protein complexes. Yet obtaining structural information on these complexes is technically difficult. Computer simulations can help to predict the geometry of a protein-protein complex and to understand the mechanism of protein association.

Here we will see two primary search strategies:

- Exhaustive search: Uses Fast Fourier Transofrmations (FFT) or surface-based techniques to scan all possible orientations.
- Stochastic Sampling: Uses Monte Carlo simulations or minimizatoin to find the optimal binding position
- Geometry-Based Docking: Fouses on **shape complentary** using FFT-based grid searches or geometric hashing.

We will be using 3 different software tools used for docking:

- FTDOCK (FTT based): uses a grid system to discretize molecules. It scores the "surface" of a static molecule (score 1) and its "core" (score -15) to find the best fit for a mobile molecule.
- ZDOCK (FTT-based): Involves marking surfaces and performing a grid-search to optimize shape complementairty.

-PatchDock (Geometric hashing): An algorithm based on the shape complementarity principles. It allows users to specify "complex types" (enzyme-inhibitor) to refine parameters.

The document outlines practical examples for running these tools:

- Example 1 (Barnase/Barstar): Typically run using ZDOCK or FTDock.
- Example 2 (Antibody/Prion): Used as a "CAPRI Target 19" example, typically run using PatchDock.
- Scoring Functions: To rank results, these tools use energy terms (electrostatics, H-bonds), conservation data, and statistical potentials.

Once docking is complete, the accuracy of the prediction is measured using three main metrics:

- F_{nat} (Fraction of native contacts): The ratio of correctly predicted contacts within $5\backslash\mathrm{AA}$ compared to the total contacts in the actual target.

- L-RMSD (Ligand RMSD): Measures the displacement of the ligand. A result is considered "near-native" if the $L - RMSD < 10\backslash\mathrm{AA}$.

- i-RMSD (Interface RMSD): Measures the root-mean-square deviation specifically at the interface residues (within $10\backslash\mathrm{AA}$).

1. Docking Barnase and Barstar using FTDock and ZDock

Barnase is an extracellular ribonuclease and barstar is an intracellular inhibitor of barnase. To stop barnase working inside the cell, barstar binds to barnase very tightly (high affinity) and very quickly (high association rate). The binding of barnase and barstar has been very well characterized experimentally in terms of structure, energetics and kinetics. The interacting surfaces have a good surface and electrostatics complementarity, so it's not surprising that the binding is very strong and fast. That makes them a perfect target for protein-protein docking studies. We will use here the unbound structures of barnase (PDB code 1A2P) and barstar (PDB code 1A19). The structure of the barnase-barstar complex is known (PDB code 1BRS), so we can compare the docking results with the experimental data.Thus, this will be our ground truth, so we could compare how the docking method perform.

We are going to use Fast fourier transformations and also geometrical hashing (to do the docking) with pathdock.

- ZDOCK and FTdock (FFtranformation)
- Pathdock (geometrical hashing)

We will use RMSD to test whether is a good or bad predictions, o compare complexes. By superimposing the native and the predicted structure and calculate the RMSD. For that we need a visualizer, either chimera or pymol to also calculate the RMSD scores. The ribonucleases are very powerful enzymes and usually the human cells synthethize it as well as an inhibitor, because otherwise all the DNA of the cells would be degradated. We have the structure of the unbound of barnase (1A2P) and barstar (1A19) and also the structure of the real complex (barnase-barstar complex = 1BRS). So we need to download the PDB:

- 1A2P --> chain A
- 1A19 --> chain A
- 1BRS, chains A, D

1.1 Prepare the input files

Firstly, download the coordinates of the unbound proteins from the protein databank, PDB codes: 1A2P and 1A19.

1.1.1 Prepare coordinates files of monomers for docking

From www.pdb.org, download coordinates of pdb code 1A2P and save them as 1a2p.pdb. DO the same for 1A19.pdb and for 1BRS.pdb. Firstly, we need to clean this file. We need to remove the lines starting with 'ANISOU' (anisotropic temperature factors, which may cause ZDOCK not to work correctly). You can use the following Linux command to make a file (1A2Patom.pdb) that keeps only the lines starting with 'ATOM'.

Here is how the structure of that specific file is organized:

- Three Symmetric Copies: The file contains three identical monomers of the Barnase protein.
- Chain Identifiers: These three copies are labeled as Chain A, Chain B, and Chain C.
- The Goal for Docking: To perform the rigid-body docking correctly, the software only needs a single representation of the protein's geometry.
- Manual Selection: The practical guide specifically tells you to edit the file to keep only the first chain A.

```
awk '$1=="ATOM" && substr($0,22,1)=="A"' 1A2P.pdb > 1a2pA.pdb
```

```
awk '$1=="ATOM" && substr($0,22,1)=="A"' 1A19.pdb > 1a19A.pdb
```

```
awk '$1=="ATOM" && substr($0,22,1)=="A"' 1BRS.pdb > 1brsA.pdb
```

```
awk '$1=="ATOM" && substr($0,22,1)=="D"' 1BRS.pdb > 1brsD.pdb
```

```
cat 1brsA.pdb > 1brsAD.pdb
cat 1brsD.pdb >> 1brsAD.pdb
```

1.2. Running ZDOCK

1.2.1 Library and script

Copy the following two files in your working directory:

```
cp /mnt/NFS_UPF/soft/zdock/IntelP3_Linux/uniCHARMM ./
cp /mnt/NFS_UPF/soft/zdock/IntelP3_Linux/create_lig ./
```

1.2.2 Formatting atomic coordinates barnase and barstar for ZDOCK

We will create new receptor and ligand pdb files (1BRS_r.pdb and 1BRS_l.pdb, respectively) with the modifications required by ZDOCK:

```
/mnt/NFS_UPF/soft/zdock/IntelP3_Linux/mark_sur 1A2P_A.pdb
1BRS_r.pdb
/mnt/NFS_UPF/soft/zdock/IntelP3_Linux/mark_sur 1A19_B.pdb
1BRS_l.pdb
```

NOTE: Disregard error messages 'Unknown type' but check that the new files have been successfully created (e.g. open with a editor). Visualize both files side by side, so you can see what is modification been made.

1.2.3 Perform the docking

Issue the following command to perform the docking `/mnt/NFS_UPF/soft/zdock/IntelP3_Linux/zdock -R 1BRS_r.pdb -L 1BRS_l.pdb -o 1BRS_zdock.out` This step takes around 5 to 10 minutes depending on the CPU load of the computer. A successful round will create a file named 1BRS_zdock.out. Open the file with your preferred editor and inspects its contents.

```
90 1.2
0.000000 0.000000 0.000000
1BRS_r.pdb 29.181 46.629 38.264
1BRS_l.pdb 92.222 58.045 3.191
1.832596 1.977582 0.969896 77 1 14 17.32
-2.094395 1.904882 -1.478089 81 2 14 16.82
-2.356194 2.104282 -0.813972 87 85 17 16.70
1.047198 2.108018 1.311430 81 88 17 15.96
[…]
```

1.2 is the resolution of the grid, and the third line is the center of mass of the protein (x and z). But we are not getting into that, no worries.

1.3. Running FTDock

1.3.1 Perform the docking

We will use the same two proteins as in ZDOCK .Use the following command to perform the FTDock docking, all in a single line:

```
ftdock -static 1A2P_A.pdb -mobile 1A19_B.pdb -noelec -
calculate_grid 1.2 -angle_step 12 -internal -15 -surface 1.3
-keep 3 -out 1BRS.ftdock
```

Note that FTDock is slightly slower than ZDock even though we have selected parameters the following parameters to speed up calculations: -calculate_grid 1.2 large grid size:1.2 (0.875 by default) and -noelec (no electrostatics; default is to use electrostatics to score poses). During FTDock execution, there are two intermediate files (scratch_scores.dat and scratch_parameters.dat) that are generated and used during the course of the execution. In case an FTDock execution is interrupted for any reason, these files are needed to restart such execution (by running ftdock –rescue). ftdock will tell which is the static, which is the structure (ligand), the gris is 1.2A which is the rotation of the grid, then the internal rotatoin and only keep 3 solution. It will calculate 500 rotations. If we type `top` we will see ftdock and/or zsock using the CPU. When zdock has finished, we will have obtained a message saying that zdock has finished. Its output should be something like this:

WARNING: if you execute two FTDock runs simultaneously from the same working directory, both executions will try to generate and use the same intermediate files, which will lead to problems.To be sure to run only one FTDock execution from the same directory at the same time, and check that no scratch_scores.dat and scratch_parameters.dat files are left in the directory before starting a new FTDock execution.

The docking round should take between 15 to 30 minutes to finish. You can monitor the progress of the docking on the terminal. A successful round will create a file named 1BRS.ftdock Open the file with your preferred editor and inspects its contents.

2. Analysis of ZDOCK and FTDOCK docking solutions

We will analyze here the docking solutions generated by FTDock and ZDOCK programs in previous two sections.

2.1. Create the pdb files from the ZDOCK docking output

We can create pdb files from the docking conformations in the ZDOCK output file. The ZDOCK output file contains information for 2000 docking solutions, so using the unmodified output would create 2000 docking files in your current directory. It is thus advisable to edit the ZDOCK output file to keep only the top N solutions (for instance, N=10, top 10). You can use a standard text editor for that, or the following linux command (forinstance, if you want to keep the top 10 solutions, i.e. N=10):

`head –n 14 1BRS_zdock.out > 1BRS_zdock_10.out` Note that we use "-n 14" to keep also the first 4 lines (in addition to the 10 lines with the top 10 solutions), which

contain important information. Now we can create pdb files for the top 10 docking solutions from the short ZDOCK output file (pdb files will be named complex.1, complex.2 and so on...). ZDOCK has a script to convert into pdb files, and create top 10 indivisdual pdb solution:

`/mnt/NFS_UPF/soft/zdock/IntelP3_Linux/create.pl 1BRS_zdock_10.out`

Note: The create lig and uniCHARMM files copied in section 1.2.1 should be in the same directory as the ZDOCK files. NOTE: If the chains are separated by an 'END' tag, perform the following(1). Otherwise, you can just rename the file

```
mv -v complex.1 complex.1.pdb
mv -v complex.2 complex.2.pdb
etc … up to
mv -v complex.10 complex.10.pdb
```

Change the format of the output files, so it can be read directly into Chimera (pdb files will be named complex.1.pdb, complex.2.pdb and so on...):

```
arrange.pl complex.1 complex.1.pdb
arrange.pl complex.2 complex.2.pdb
arrange.pl complex.3 complex.3.pdb
arrange.pl complex.4 complex.4.pdb
arrange.pl complex.5 complex.5.pdb
arrange.pl complex.6 complex.6.pdb
arrange.pl complex.7 complex.7.pdb
arrange.pl complex.8 complex.8.pdb
arrange.pl complex.9 complex.9.pdb
arrange.pl complex.10 complex.10.pdb
```

Because teh ZDOCK format is slightly different, so better do the previous step.

2.2. Create the pdb files from the FTDock docking output

The same way we chose top 10 from ZDOCK, the same for FTDock. We can create pdb files from the FTDock output file:

```build -in 1BRS.ftdock -b1 1 -b2 10```

This command will create pdb files for the top 10 docking solutions (from 1 to 10) in the FTDock output file (pdb files will be named Complex_1g.pdb, Complex_2g.pdb, etc).

Now we have all the information that we need, have rpepared the coordinates, have done the docking at took top 10 solutions against 1BRS_AD. Now compare this with the real strucutres to retrieve the RMSD values. Superimpose 1BRS with 1A19 and 1A2P.
```

2.3. Visualization of docking solutions

The objective is to compare the top 10 dockings poses of both FTDock and ZDOCK and compare with the reference complex: the solved Xray structure of the Barnase/Barstarcomplex you prepared earlier: 1BRS_AD.pdb. In the previous sections you have run the docking and extracted the top 10 docking poses of both docking programs. Making use of chimera you should compare these docking poses with real complex. You will have to load the docking poses and the Xray complex in chimera* and perform the structural superposition using the Barnase as reference.

Consider the following question in your analyses:

- **How close are the highest-scoring FTDock and ZDOCK solutions to the x-ray structure? (near = less than 10A)** The "highest-scoring" solution is the one ranked #1 by the program's scoring function.
- ZDOCK: The highest-scoring solution (complex.1) has an RMSD of $10.142\backslash\mathrm{AA}$. This is technically just above the "near-native" threshold ($10\backslash\mathrm{AA}$).
- FTDock: The highest-scoring solution (Complex_1g) has an RMSD of $15.377\backslash\mathrm{AA}$, which is far from the native x-ray structure.
- **How many docking solutions with RMSD<10Å can be found in the top 10 solutions from each program?** The PDF defines a "near-native" prediction as having an $L - RMSD < 10\backslash\mathrm{AA}$.
- ZDOCK: You found 5 solutions (Complexes 10, 2, 3, 7, and 9).
- FTDock: You found 1 solution (Complex_10g).
- **Will the combination of results from both programs improve the chances of a good prediction?**

Yes, combining results generally improves the chances of a good prediction for several reasons:

Methodological Diversity: ZDOCK and FTDock use different grid-based scoring functions and discretization methods.

Consensus Ranking: If a specific binding pose is identified as high-scoring by multiple independent algorithms, it increases the confidence that the pose is biologically relevant rather than a computational artifact.

Sampling Coverage: In your results, ZDOCK was much more successful at finding near-native poses, but FTDock's top-scoring result was actually different from ZDOCK's. Using both ensures that if one algorithm fails to find a "hit" due to its specific scoring parameters, the other might succeed.

- From this analysis on a test case in which complex structure is known, have we learned any suggestion to optimally apply these docking programs in a real case situation?
- **Identify the closest docking pose among the top 10 solution from each program to the real complex.**

Closest ZDOCK Pose: complex.3 with an RMSD of $5.844\backslash\mathrm{AA}$.

Closest FTDock Pose: Complex_10g with an RMSD of $9.951\backslash\mathrm{AA}$.

Note. You can also use Pymol to structurally align and compute RMSD values to analyze your docking solutions. In the campus, you can find a quick 'how-to' to achieve this with pymol (quick how to structural superposition pymol.pdf)

In a real scenario, we don't know at first which is the complex, so first relay on the score. Superimpose and take clusters telling which region of the surface is more populated and migth tell you which is the real interacting region. Based on this model, make mutations experimetnally and infer if the model is true or false.

activate conda pymol and type pymol. And open the native complex (1BRS_AD) and in action we can change the color of the chain, show surface if we want to (and if not hide the surface). now load the docking solutions, for instance, use the ones from FTDock, dock all of them plugin > alignment > in method superimpose , uncheck outlier rejection, mobile selection = *, target selection = 1brs_ad, and it will do the superomposition. And tells you the RMSD of each superimposition. The lowest score, the the better, and in this case the best complex is the 10th. If we only take into account chain A, it will force somehow to superimpose perfeeclty, while having another chain will be more realistic.

Now do the same but for zdock, by opining another pymol session. Load the reference (BRS_AD) and the complex output for the zdock. Again, plugin > superimpose > many-to-one (same as before). What happens here, just by looking at scores, xdock has more near solutions among top10, yields better scores (lower scores). So, both sampling solutions are comprehensive, getting a very good samping among all docking space, but zdock is better at scoring them, because it is promoting the good ones among the top.

Now use patch dock which uses geometrical hashing:

3. Docking of the ovine prion protein and a neutralizing antibody using

PathDock

Now we will use PatchDock, a useful and popular rigid-body docking program that is based on the identification of surface complementarity between the two interacting proteins. We will perform the docking of an antibody to the ovine prion. Prion diseases are deadly neurodegenerative pathologies affecting numerous mammal species [1]. The key event in the pathogenesis is the conversion of the α-helix-rich host prion protein (PrPC) into a pathogenic isoform (PrPSc) characterized by its insolubility, its high content in β-sheet, and its protease resistance.

3.1. Input files for docking

Download the SBI_15_P.zip from the Aula global. It contains the following files:

- antibody.pdb - atomic coordinates of the antibody
- ovine.pdb – atomic coordinates of the ovine prion: antigen
- NativeComplex.pdb – atomic coordinates of the Xray complex (1TPX)

3.2 Running PatchDock

Is even simplier to run than zdcok and ftdock.

3.2.1 Create the parameter file that is required by PatchDock

We need to define some paramters, 4.0 will be cut-off of the clustering, AA is type of protein complex. Patchdock ahas 4 types of protein complex, AA = antigen-antobody complex.

`buildParams.pl antibody.pdb ovine.pdb 4.0 AA`

This step creates a text file: params.txt with all the parameters required to run PatchDock. First and second arguments are the input atomic coordinates in the form of receptor and ligand, RMSD cut-off clustering and type of complex. In this case is 'AA' which stands for Antigen-Antibody complex. There are other types considered by PatchDock: EI: Enzyme/Inhibitor; drug: protein/ligand; default: protein/protein (neither AA nor EI). The params file tells teh programs which paramters has to take into account.

3.2.2. Perform the docking

Issue the following command in a single line `patch_dock.Linux params.txt out_file1` PathDock is very fast and the docking will be done in a few minutes. The results will be store in the file: out_file1. Open the file with a text editor and analyze the format of the file: does the geometrical hashing o bot proteins, seeing if tehre's a cavity, a protution.

at the beginning will be the parametrs that patchdock hase used:

```
*********************************************************
Program parameters
ACE_EnergyTermWeight (Str) 1.0
COM_distanceTermWeight (Str) 1.07
HBEnergyTermWeight (Str) 1.0
attrVdWEnergyTermWeight (Str) 1.01
baseParams (Str) 4.0 13.0 2
clusterParams (Str) 0.1 4 2.0 4.0
confProbEnergyTermWeight (Str) 0.1
desolvationParams (Str) 500.0 1.0
elecEnergyTermWeight (Str) 0.1
energyDistCutoff (Str) 6.0
ligandActiveSite (Str) site2.txt
ligandGrid (Str) 0.5 6.0 6.0
ligandMs (Str) carly1.pdb.ms
ligandPdb (Str) carly1.pdb
ligandSeg (Str) 10.0 20.0 1.5 1 1 1 1
log-file (Str) patch_dock.log
log-level (Str) 2
matchAlgorithm (Str) 1
matchingParams (Str) 1.5 1.5 0.4 0.5 0.9
piStackEnergyTermWeight (Str) 0.0
protLib (Str) /home/narcis/DOCKINGS_CD19/PatchDock/chem.lib
radiiScaling (Str) 0.8
receptorActiveSite (Str) site1.txt
receptorGrid (Str) 0.5 6.0 6.0
receptorMs (Str) CD19.pdb.ms
receptorPdb (Str) CD19.pdb
receptorSeg (Str) 10.0 20.0 1.5 1 1 1 2
repVdWEnergyTermWeight (Str) 0.5
scoreParams (Str) 0.3 -5.0 0.5 0.0 0.0 1500 -8 -4 0 1 0
vdWTermType (Str) 1
*********************************************************
```

obtain something like this:

```
# | score | pen. | Area | as1 | as2 | as12 | ACE | hydroph |
Energy |cluster| dist. || Ligand Transformation
1 | 11502 | -3.26 | 1623.00 | 2 | 11290 | -3.78 | 1525.60 |
1797 | 2331 | 2106 | 1821 | 683 | 313.12 | 1146.31 | 1333 |
191.56 | 1013.13 | 0.00 | 0.00 | 0 | 0.00 || -0.05752 1.19704
-2.20131 -37.29977 -2.48074 -3.56842
0 | 0.00 || 0.04000 0.12658 -2.77974 -34.22400 -16.34312
3.45601
[…]
```

3.2.3 Generate the top 10 docking solutions

To generate the top 10 docking solutions simply issue the following command in a single line: `transOutput.pl out_file1 1 10`

We will get out_file1.pdb, out_file2.pdb, and so on and so forth.

3.3 Visualization and analysis of the PatchDock results

Compare the docking solution with the native structure. As in the previous exercise open the top 10 docking solutions and the reference complex. Superimpose all the structures and analyze the results. Are the results reasonable/accurate?

yes, almost 50% of them have quite good lower scores.

Which docking poses among the top 10 is the closest to the reference complex?

the 10th would be the closer to the native strucutre, as it is the one that has lower scores.